FUTUREWAY
KI INSTITUT

Modul 1: KI-Grundlagen

Lektion 9: APIs & MCP

KI Automations Spezialist

Juli 2026

Du nutzt APIs längst — ohne es zu merken

☁️ Wetter

Echtzeitdaten vom Server auf dein Display

🔑 Google Login

„Mit Google anmelden" — eine API im Hintergrund

💳 Bezahlen

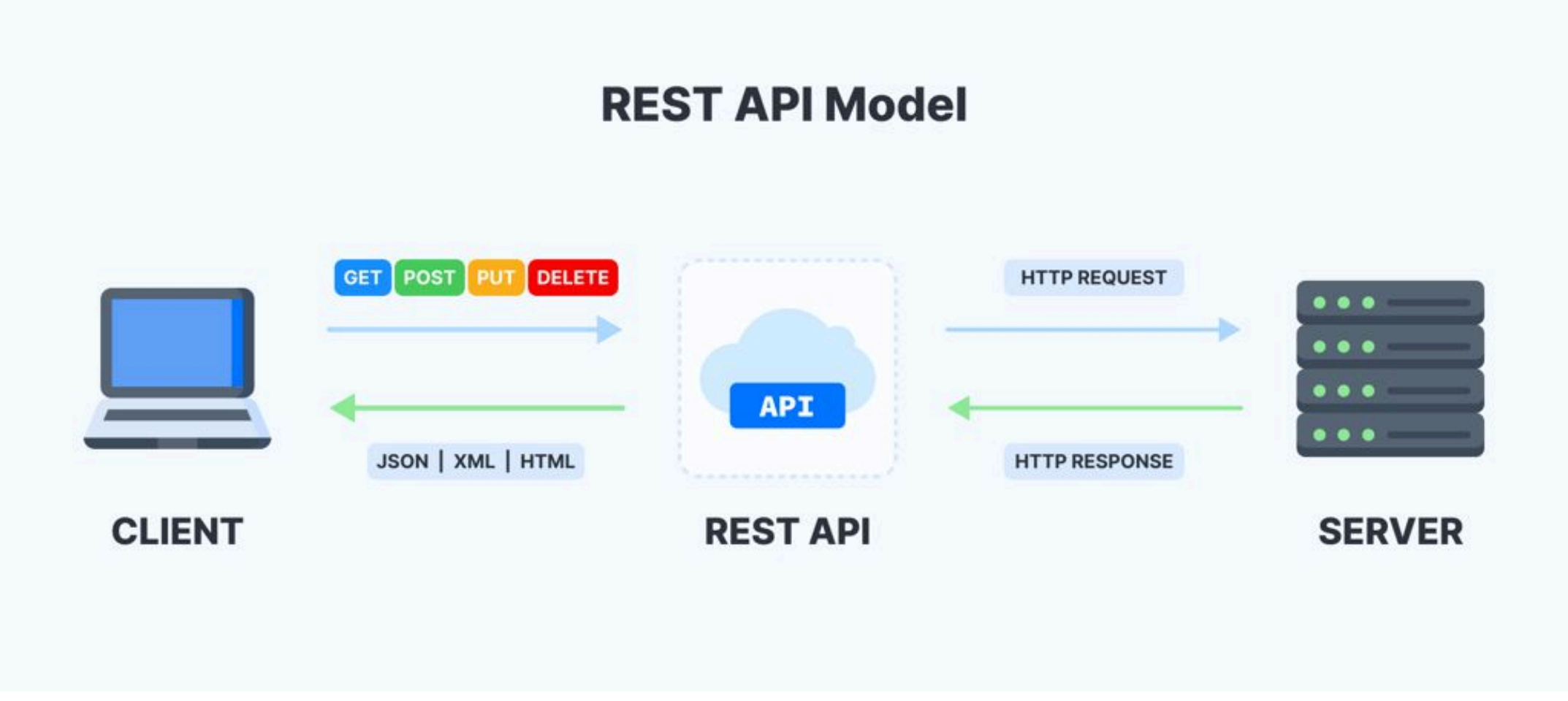
Stripe, PayPal & Co. sind nichts als APIs

🗺️ Navigation

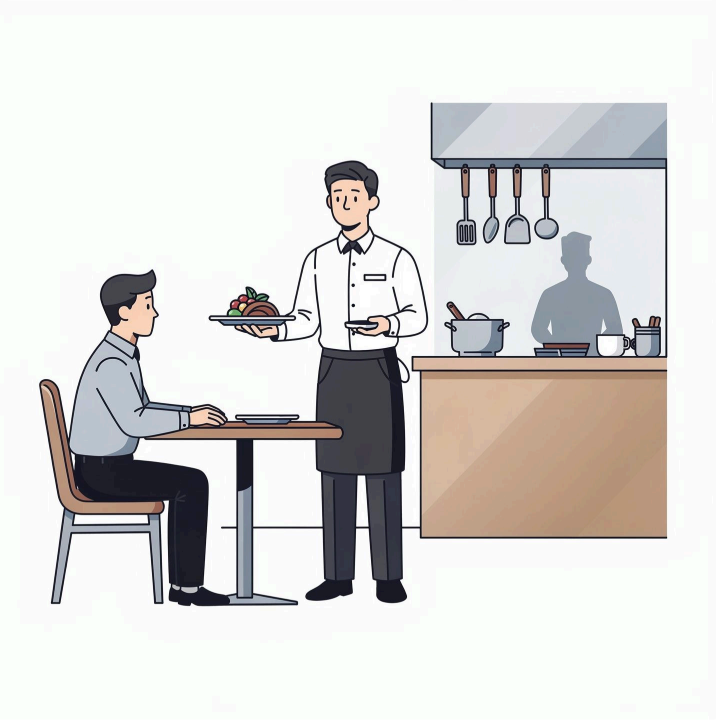
Google Maps liefert Routen via API

APIs verstehen

Wie Software miteinander spricht — ganz einfach erklärt



Wie ein Kellner im Restaurant



Du bestellst — die Küche kocht — du musst nicht wissen, wie.

App

Der Gast am Tisch — stellt Anfragen

API

Der Kellner — übersetzt und vermittelt

Server

Die Küche — verarbeitet und liefert

Es gibt verschiedene Standards

Im Web hat sich ein klarer Favorit durchgesetzt — **REST** ist der mit Abstand meistgenutzte Standard.

1

SOAP

XML, schwergewichtig, Enterprise (eher veraltet)

z. B. ELSTER

2

REST ⭐

Standard im Web (HTTP + JSON)

z. B. Stripe, GitHub

3

GraphQL

Client bestimmt die Daten; ideal für Dashboards

z. B. Shopify

4

Webhook

Server meldet sich selbst bei Events

z. B. neue Bestellung

5

gRPC

Sehr schnell, binär; für Microservices

z. B. Netflix

6

WebSocket

Dauerhafte Live-Verbindung; Echtzeit

z. B. Live-Chat

Wir schauen uns **REST** genauer an — es ist der Standard im modernen Web.

Was passiert wirklich?

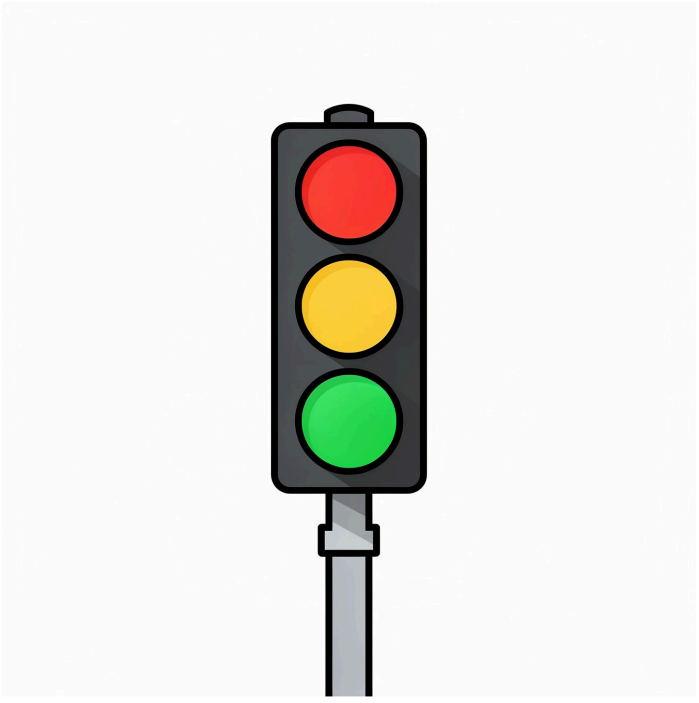
📨 Die Frage (Request)

> „Gib mir das Wetter in Berlin"

📦 Die Antwort (Response)


```
{ "stadt": "Berlin",
  "temp": 25,
  "wetter": "sonnig" }
```

Der Server antwortet mit strukturierten Daten — meistens im JSON-Format. Lesbar für Maschinen *und* Menschen.



REST-API

Hat's geklappt?

Jede API-Antwort enthält einen **Statuscode** — wie eine Ampel für den Erfolg deiner Anfrage.

200 – Alles gut ✅

Anfrage erfolgreich, Daten kommen zurück

404 – Nicht gefunden ❓

Die Ressource existiert nicht

500 – Serverfehler ⚠️

Etwas ist auf dem Server schiefgelaufen



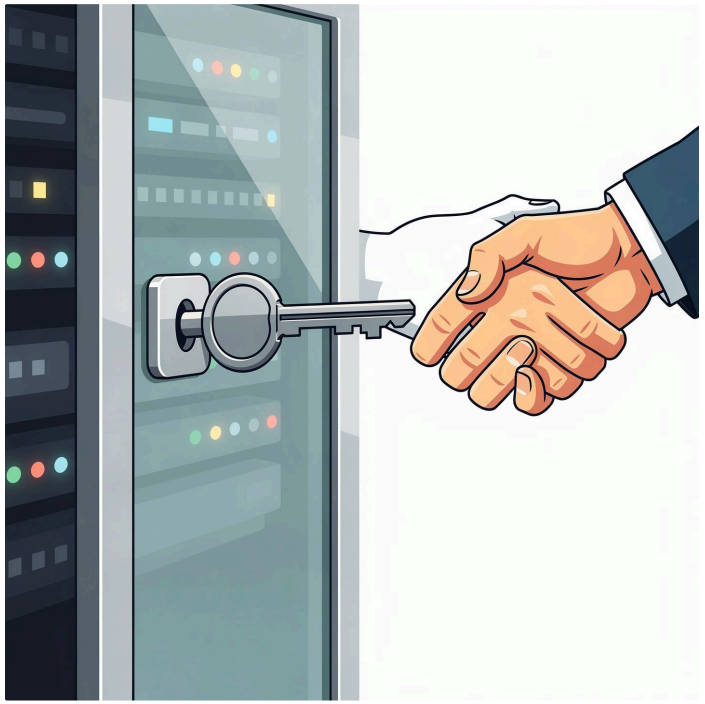
Nicht unbegrenzt: Limits & Kosten

⏳ Rate Limit

Nur **X Anfragen pro Minute** erlaubt. Wer zu viel fragt, bekommt Fehler **429**.

💶 Kosten pro Anfrage

Viele APIs — besonders KI-APIs wie **OpenAI** — werden pro genutztem Aufruf abgerechnet.



Wer darf rein?

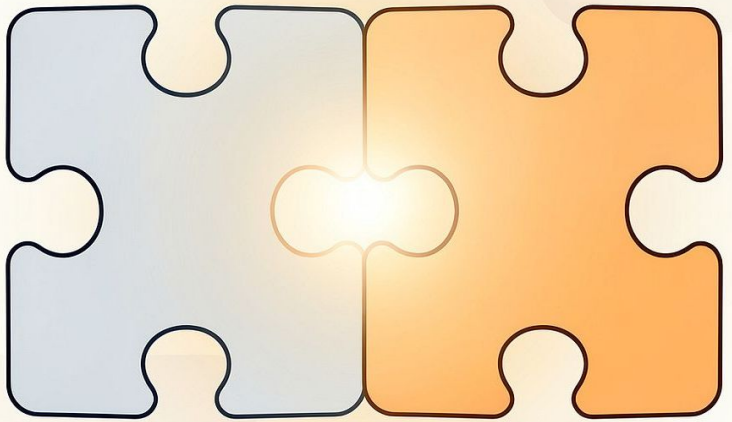
APIs schützen sich durch Authentifizierung — zwei Methoden dominieren die Praxis.

API-Key

Dein persönlicher Zugangscode. Einfach einzurichten — aber **niemals teilen**.

OAuth 2.0

Zugriff erlauben, *ohne* Passwort zu teilen. Das „Mit Google anmelden"-Prinzip.



„Wer APIs versteht, kann Systeme verbinden."

APIs sind die **Sprache des Internets**

Model Context Protocol

Model Context Protocol – der „USB-C-Anschluss" für KI



APIs verbinden Software mit Software

Grundlegend für den Datenaustausch zwischen Anwendungen.



MCP verbindet KI-Modelle & Agenten einheitlich mit Tools, Daten & APIs

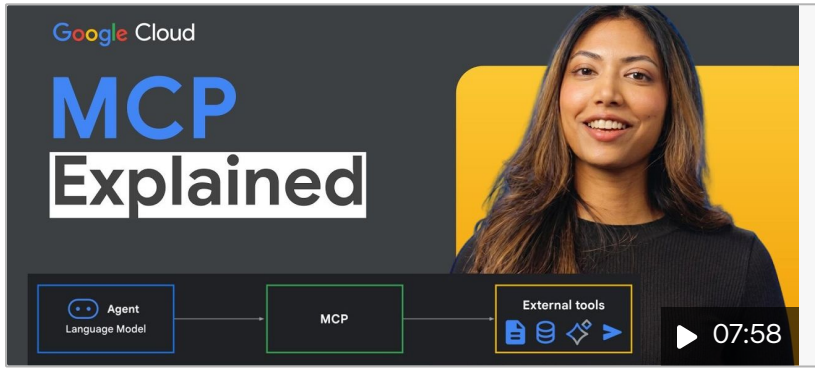
Einheitliche Schnittstelle für komplexe KI-Ökosysteme.



Ein Standard statt tausender Einzel-Integrationen

Vereinfacht die Entwicklung und Skalierung von KI-Anwendungen.

Funktionsweise MCP

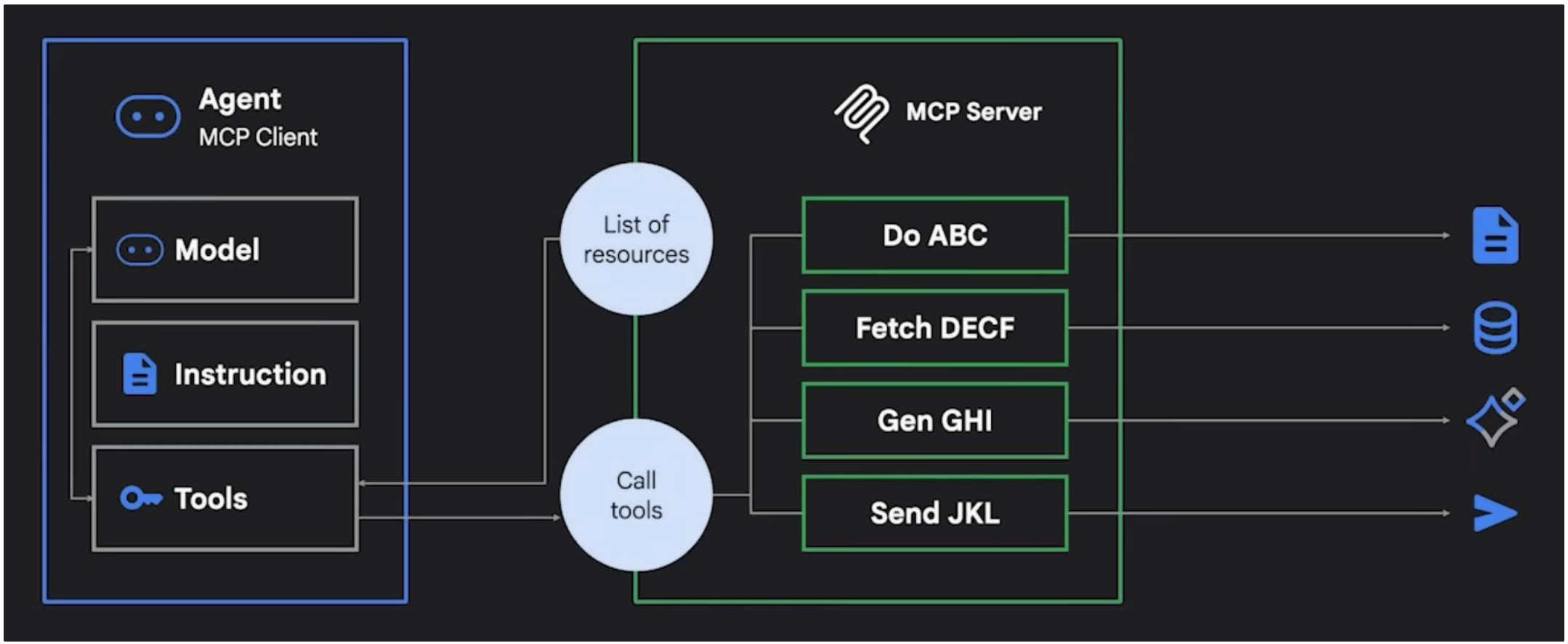


YouTube

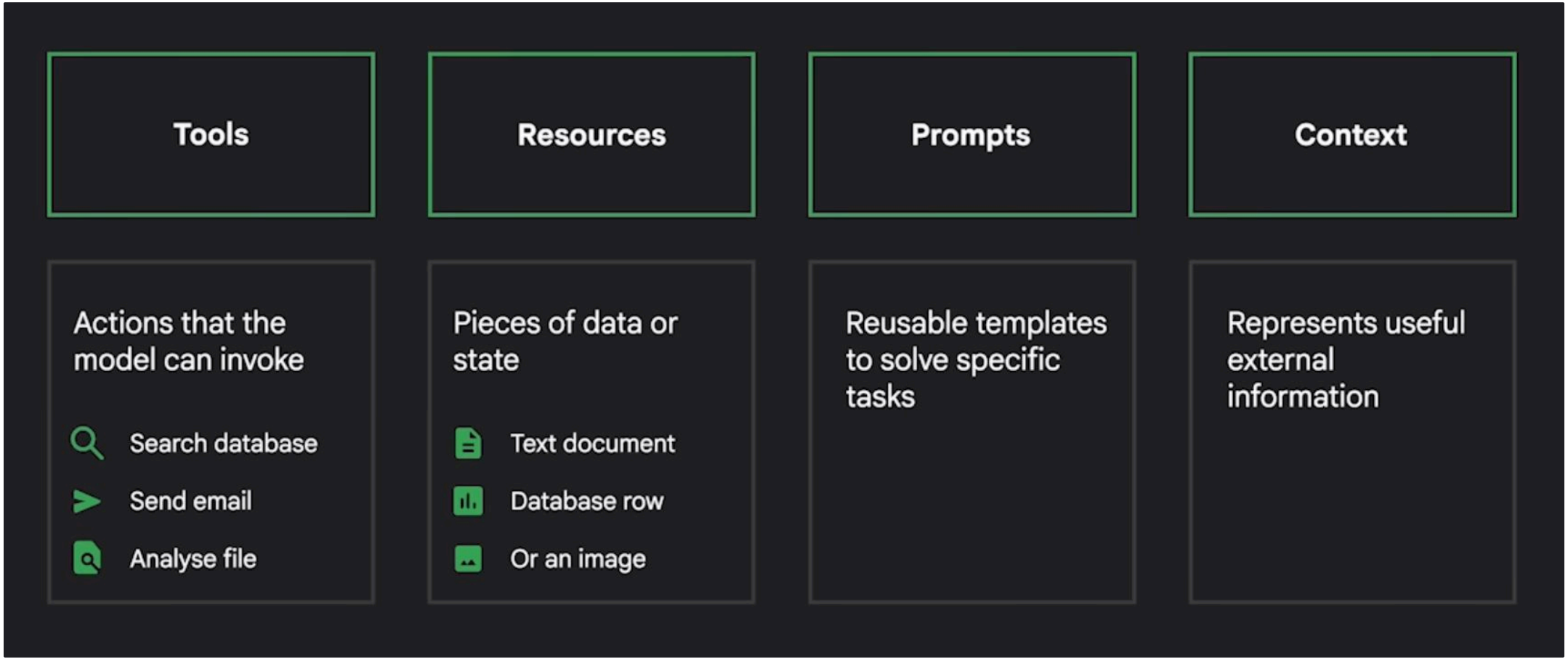
How Model Context Protocol (MCP) actually works

Google MCPs → https://goo.gle/3PYkjky Build an ADK agent with Google MCPs → https://goo.gle/4o6h8DP Connect an MCP server to an AI agent →...

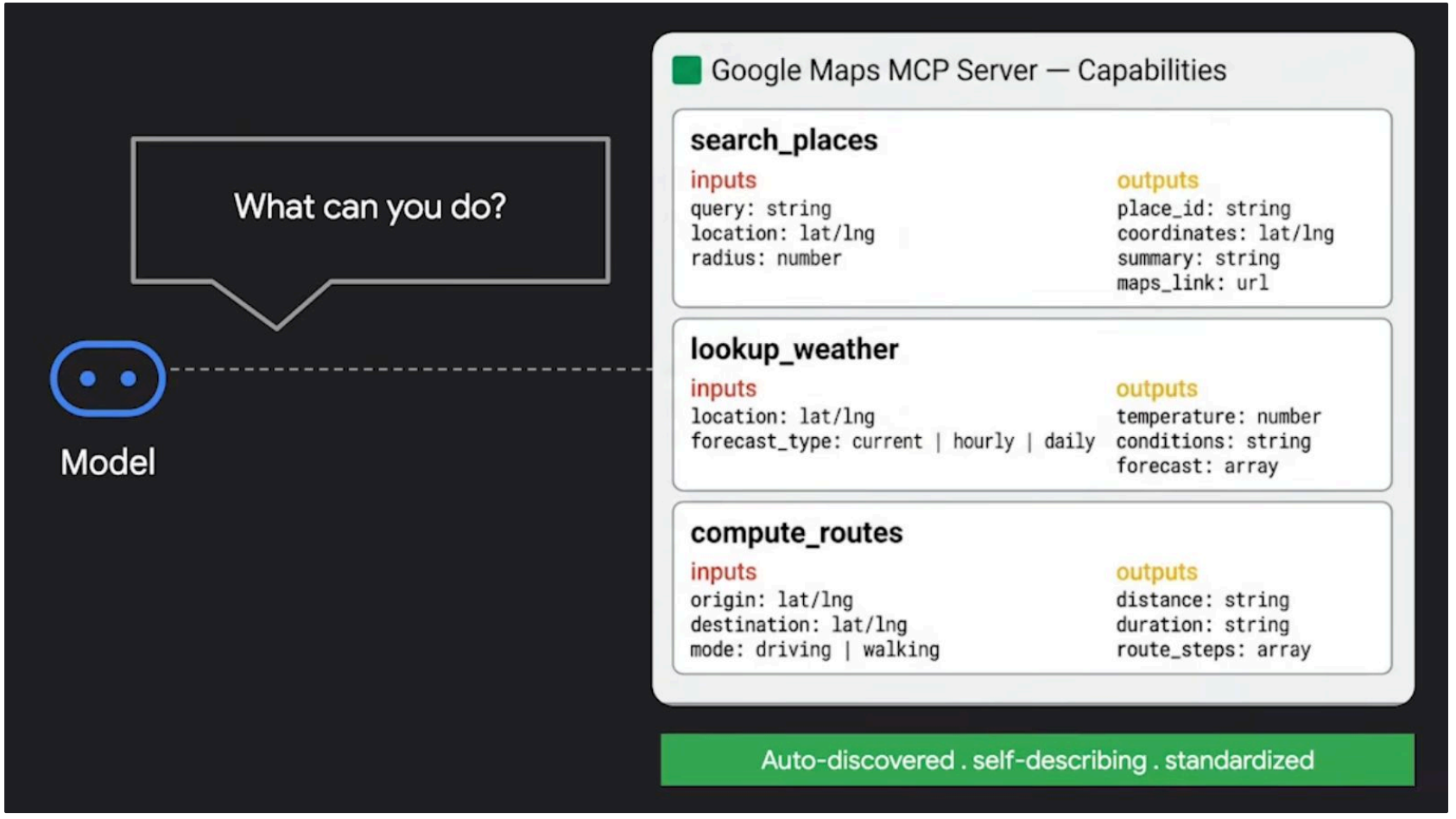
Wie funktioniert MCP?



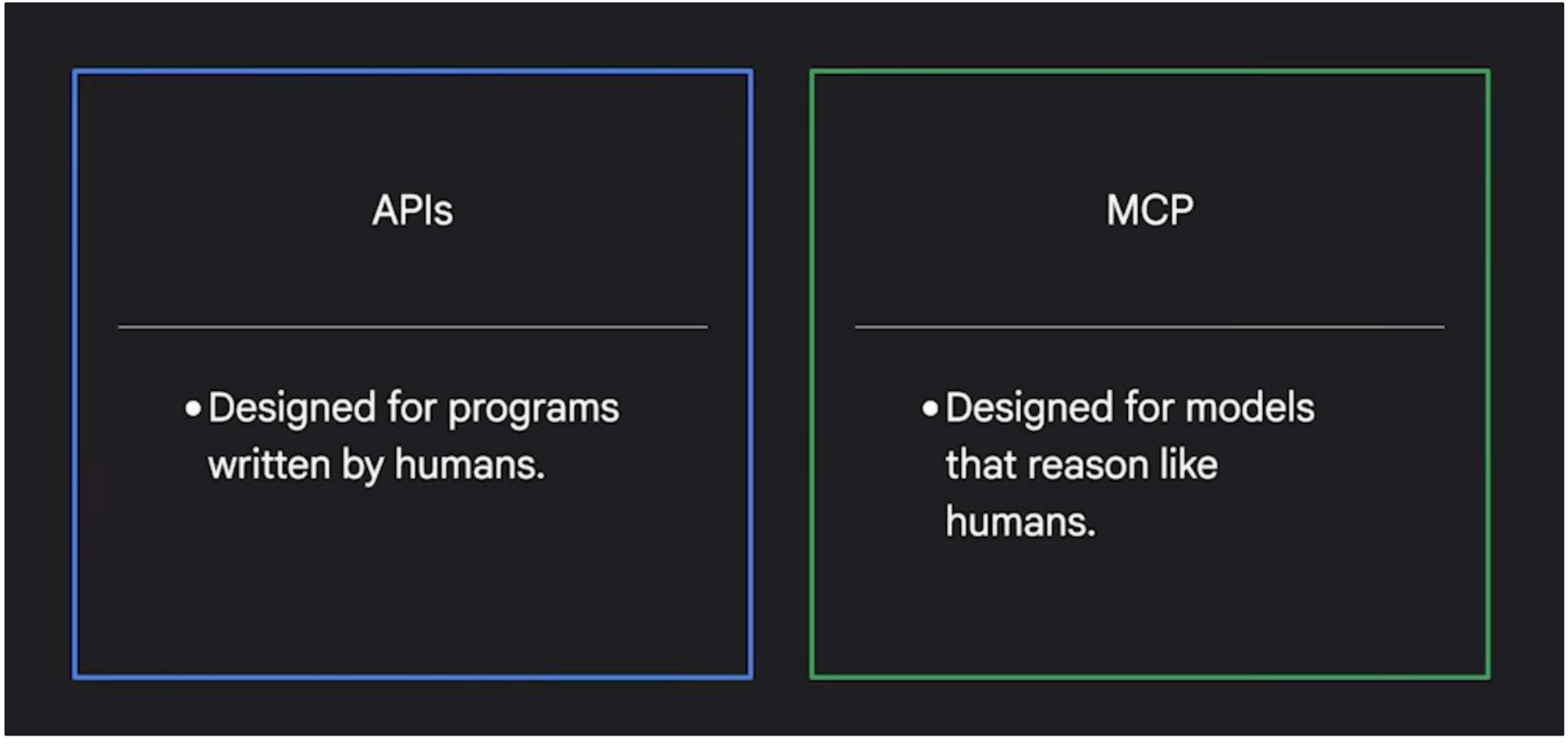
Welche Daten tauscht MCP aus?



Beispiel: Google Maps MCP

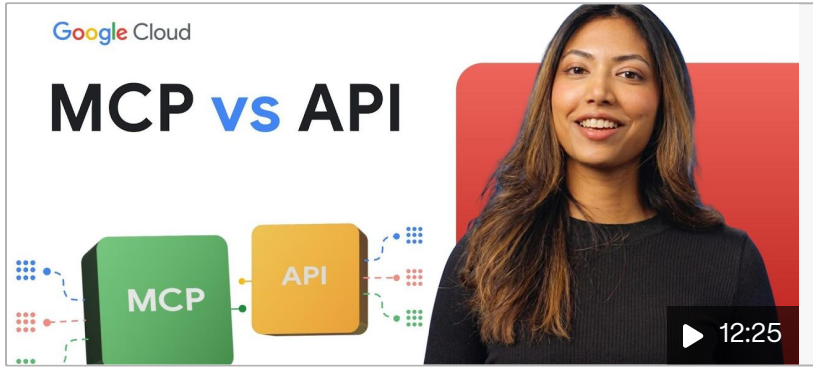


Warum nicht einfach eine API für agentische KI nehmen?



Im Hintergrund werden immer noch API-Requests verwendet. Der MCP-Server ist also ein Abstraktionslayer, der APIs für Agenten nutzbar macht.

MCP vs API



YouTube

MCP vs API: Why traditional APIs are failing AI agents

MCPs explained → https://goo.gle/4wjeLka Google MCP repo → https://goo.gle/4aeGUA5 Model Context Protocol (MCP) → https://goo.gle/4uQk3D...

REST APIs

Was ist eine REST API?

REST (Representational State Transfer) ist ein Architekturstil, der festlegt, wie Anwendungen über HTTP miteinander kommunizieren sollen.

Zustandslos

Jede Anfrage enthält alle nötigen Informationen – der Server merkt sich zwischen zwei Anfragen nichts.

Ressourcenorientiert

Ressourcen (z.B. Bücher) stehen im Mittelpunkt und werden über eindeutige URLs angesprochen.

JSON-Format

Antworten meist im JSON-Format – leicht lesbar für Maschinen und Menschen.

HTTP-Methoden (Request Type)



GET

Daten **holen** — lesen ohne zu verändern



POST

Neues **anlegen** — Datensatz erstellen



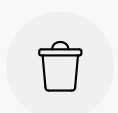
PUT

Vorhandenes **ändern** — aktualisieren



PATCH

Teilweise **ändern** — nur einzelne Felder aktualisieren



DELETE

Eintrag **löschen** — entfernen

CRUD

Hinter den HTTP-Methoden steckt ein einfaches Konzept — CRUD:

Kürzel	Operation	HTTP-Methode
C	Create — Erstellen	`POST`
R	Read — Lesen	`GET`
U	Update — Aktualisieren	`PUT` / `PATCH`
D	Delete — Löschen	`DELETE`

CRUD beschreibt die vier Grundoperationen jeder Datenverwaltung — APIs setzen sie mit HTTP-Methoden um.

Beispiel: Buchladen

Die API eines Online-Buchladens ist unter `https://api.buchladen.de/v1` erreichbar.



Ressourcen

In einer REST API sind **Ressourcen** die zentralen "Objekte" oder "Datenentitäten", auf die du zugreifen oder die du manipulieren möchtest. Sie sind die "Nomen" deiner API.

Bücher Ressource: `/buecher`

Stellt eine Sammlung von Büchern dar, auf die du zugreifen kannst.

- Alle Bücher abrufen: `GET /buecher`
- Ein spezifisches Buch: `GET /buecher/42`
- Neues Buch hinzufügen: `POST /buecher`

Kunden Ressource: `/kunden`

Stellt eine Sammlung von Kundendaten dar, die verwaltet werden können.

- Alle Kunden abrufen: `GET /kunden`
- Ein spezifischer Kunde: `GET /kunden/123`
- Neuen Kunden anlegen: `POST /kunden`

Ressourcen werden über eindeutige URLs identifiziert und sind der Kern jeder RESTful API.

HTTP-Methoden (Request Type)



GET

`/buecher` – Bücher **abrufen** (lesen ohne zu verändern)



POST

`/buecher` – ein neues Buch **anlegen** (Datensatz erstellen)



PUT

`/buecher/42` – ein Buch komplett **ersetzen** (aktualisieren)



PATCH

`/buecher/42` – einzelne Felder eines Buchs **ändern** (partielle Aktualisierung)



DELETE

`/buecher/42` – ein Buch **löschen** (entfernen)

GET – Bücher abrufen (mit Filter)

Request – Anfrage

Methode	GET
URL	https://api.buchladen.de/v1/buecher?autor=Goethe&jahr=1808
Parameter	• Autor: Goethe • Jahr: 1808
Athorization	Bearer *bl-55eb1ce7653e4e2dafa62b1e91c06f17*
Body	-

Response – Antwort

Antwort – `200 OK`, alle passenden Bücher als Liste

```
[
  {
    "id": 42,
    "titel": "Faust",
    "autor": "Johann Wolfgang von Goethe",
    "jahr": 1808
  }
]
```

GET – Ein spezifisches Buch per ID abrufen

Request – Anfrage

Methode	GET
URL	https://api.buchladen.de/v1/buecher/42
Parameter	-
Athorization	Bearer *bl-55eb1ce7653e4e2dafa62b1e91c06f17*
Body	-

Response – Antwort

Antwort – `200 OK`, genau ein Buch


```
{
  "id": 42,
  "titel": "Faust",
  "autor": "Johann Wolfgang von Goethe",
  "jahr": 1808,
  "preis": 12.99
}
```

POST - Ein Buch anlegen

Request - Anfrage

Methode	POST
URL	https://api.buchladen.de/v1/buecher
Parameter	-
Athorization	Bearer *bl-55eb1ce7653e4e2dafa62b1e91c06f17*

Body	{
	"titel": "Die Leiden des jungen Werthers",
	"autor": "Johann Wolfgang von Goethe",
	"jahr": 1774,
	"preis": 8.00
	}

Response - Antwort

Antwort – `201 Created`, der Server vergibt eine neue `id`

{
"id": 43,
"titel": "Die Leiden des jungen Werthers",
"autor": "Johann Wolfgang von Goethe",
"jahr": 1774,
"preis": 8.00
}

PUT – Ein Buch komplett ersetzen

PUT ersetzt das **ganze** Objekt. Es müssen **alle** Felder mitgeschickt werden – was fehlt, wird überschrieben (gelöscht).

Request - Anfrage

Methode	PATCH
URL	https://api.buchladen.de/v1/buecher/42
Parameter	-
Athorization	Bearer *bl-55eb1ce7653e4e2dafa62b1e91c06f17*

Body	{
	"titel": "Faust. Eine Tragödie",
	"autor": "Johann Wolfgang von Goethe",
	"jahr": 1808,
	"preis": 12.99
	}

Response - Antwort

Antwort – `200 OK` mit dem vollständig ersetzten Buch


```
{
  "id": 42,
  "titel": "Faust. Eine Tragödie",
  "autor": "Johann Wolfgang von Goethe",
  "jahr": 1808,
  "preis": 12.99
}
```

PATCH – Einzelne Felder ändern

PATCH ändert nur die **mitgeschickten** Felder – der Rest bleibt unverändert. Ideal für kleine Anpassungen wie einen neuen Preis.

Request - Anfrage

Methode	PATCH
URL	https://api.buchladen.de/v1/buecher/42
Parameter	-
Athorization	Bearer *bl-55eb1ce7653e4e2dafa62b1e91c06f17*

Body	{
	"preis": 9.99
	}

Response - Antwort

Antwort – `200 OK`, nur der Preis hat sich geändert

{
"id": 42,
"titel": "Faust. Eine Tragödie",
"autor": "Johann Wolfgang von Goethe",
"jahr": 1808,
"preis": 9.99
}

DELETE – Ein Buch löschen

DELETE entfernt eine Ressource. Der Body bleibt leer.

Request - Anfrage

Methode	DELETE
URL	https://api.buchladen.de/v1/buecher/43
Parameter	-
Athorization	Bearer *bl-55eb1ce7653e4e2dafa62b1e91c06f17*
Body	-

Response - Antwort

Antwort – `204 No Content`: erfolgreich gelöscht, es gibt nichts zurückzugeben.

Statuscodes verstehen

Der Statuscode sagt sofort, wie die Anfrage ausgegangen ist – gruppiert nach Hunderterbereichen:

2XX

Erfolg

- `200 OK` (Buch geladen)
- `201 Created` (Buch angelegt)
- `204 No Content` (Buch gelöscht)

4XX

Fehler beim Client

- `404 Not Found` (Buch nicht gefunden)
- `401 Unauthorized` (Token fehlt oder ist ungültig)

5XX

Fehler beim Server

- `500 Internal Server Error` (Serverseitiger Fehler)

Faustregel: 4xx-Fehler liegen an der Anfrage des Clients, **5xx**-Fehler am Server.

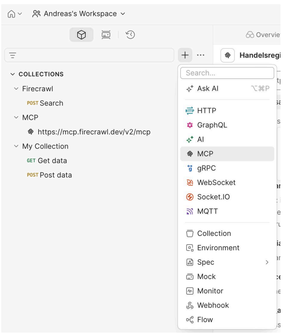
Aufagbe: Lead-Anreicherung (Unternehmensrecherche)

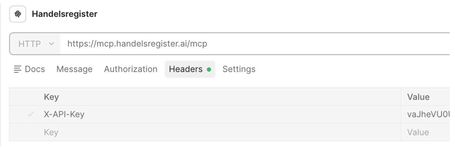
Part 1 - Agent aufbauen & testen

1. Agent Template "Unternehmensrecherche" in Langdock duplizieren
2. Ausführen und Testen
3. Erforderliche Anpassungen machen

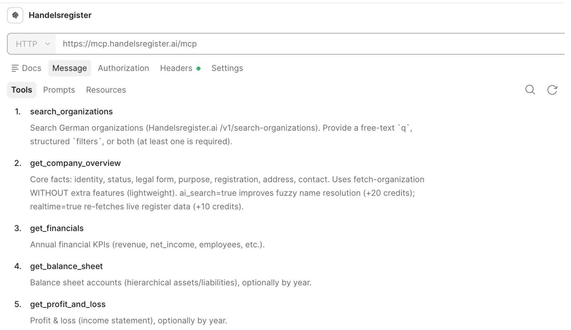
Part 2 - Handelsregsiter.io MC Server integrieren

1. Gehe zu **https://handelsregister.ai/** und erstelle dir einen Account, indem du auf "Registrieren" klickst
2. Kopiere den API-Key
3. Gehe zu Integrationen in Langdock → suche nach "Handelsregister.io" → " + Integration hinzufügen" → vergib einen aussagekräftigen Namen für die Verbindung (z. B. "Handelsregister <dein Vorname>") → füge den API-Key ein → teste die Verbindung
4. Mache dich mit der Funktionalität des MCP Servers von **handelsregister.ai** vertraut. Gehe dazu auf deinen Postman Workspace (**postman.co**) und verbinde dich mit dem MCP Server (**https://mcp.handelsregister.ai/mcp**). Gehe dazu nach untenstehenden Screenshots vor. Füge zur Authentifizierung den Header mit Key "X-API-Key" hinzu (Beschreibung in Screenshot).

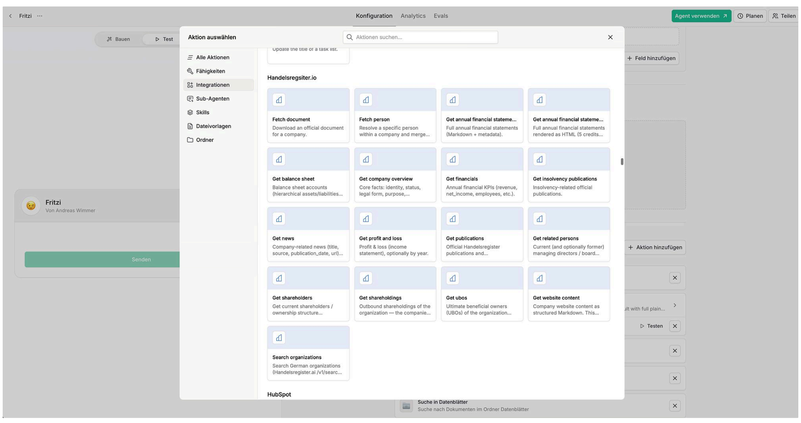




5. Unter "Message" findest du alle verfügabren Tools des MCP Servers.



6. Diese Tools werden wir weiter für unseren Agent verwenden (nicht zu viele, um Credits zu sparen):
   a. get_company_overview
   b. get_financials
   c. get_news
   d. get_website_content
7. Gehe in Langdock in den Editor-Modus deines Agents und gehe zu "Aktionen" > "+ Aktion hinzufügen" > "Integrationen". Scrolle dann zu "Handelsregister.io" und wähle die benötigten Tools aus.



8. Passe final den System Prompt an, sodass die Tools vom Agent verwendet werden. Integriere dazu die expliziten Tool-Namen, wie "get_company_overview" in den System Prompt des Agent. Sei möglichst explizit, welche Parameter in den Tool-Calls verwendet werden sollen und welche vom Agent definiert werden sollen. Beispielsweise würde ich empfehlen, bei "get_company_overview" "ai_search" zu deaktivierten, um Credits zu sparen. Schreibe dazu in den System-Prompt: "get_company_overview (ai_search:false)".
9. Passe den System Prompt nochmals final an, sodass die Agent-Vorlage durch die neuen Daten aus **handelsregister.ai** sinnvoll erweitert wird.

Part 3 - Testing und Fine-Tuning

1. Erstelle eine Abfrage über ein dir bekanntes Unternehmen deiner Wahl und überprüfe den Output.
2. Falls Verbesserungsbedarf am Output besteht, muss möglicherweise der System Prompt angepasst werden.
3. Wiederhole solang, bis das gewünschte Ergebnis erreicht wird. Behalte dabei die verbrauchten Tokens und Credits des MCP Servers im Auge.

Bonus 1: DOCX Vorlage Unternehmensprofil

Ertelle eine Docx Vorlage für das Unternehmensprofil, das dein Agent verwendet, um das Unternehmensprofil aufbereitet darzustellen. Stelle sicher, dass immer das gleiche Format, die gleiche Struktur und das gleiche Styling verwendet werden. Wir wollen am Ende reproduzierbare Ergebnisse.

Bonus 2: Versand des Unternehmensprofils per Mail

Das DOCX File wird automatisiert an bestimmte Empfänger über die Outlook-Integration versandt.